CS 422 HW3 Report: eBPF Congestion Control (Option 1)

Repository: CS422-Networks-Group/HW3 (branch `hw3-cleanup`). Raw measurements: `results/`. Dependencies and reproduction: README.

We implemented our HW2 Q3(b) algorithm, **dgcc** (delay + goodput congestion control), as an eBPF `struct_ops` `tcp_congestion_ops` program (`bpf/dgcc.bpf.c`) and compared it with CUBIC, Reno, and BBR using our HW2 Python client (`src/main.py`).

1. Kernel, required features, build and load

Kernel	`6.6.87.2-microsoft-standard-WSL2` (Ubuntu 24.04.2 on WSL2, x86_64)
Required features	`CONFIG_BPF_SYSCALL=y`, `CONFIG_BPF_JIT=y`, `CONFIG_DEBUG_INFO_BTF=y`; kernel ≥ 5.6 for BPF congestion control; `tcp_bbr` module for the BBR baseline
Toolchain	Ubuntu clang 18.1.3, libbpf-dev 1.3.0, bpftool from `linux-tools-generic` 6.8.0-147

```
sudo apt install -y clang llvm libbpf-dev linux-tools-common linux-tools-generic make python3-venv
make -C bpf             # build bpf/dgcc.bpf.o
sudo make -C bpf load   # register dgcc, load tcp_bbr, allow all four algorithms
cat /proc/sys/net/ipv4/tcp_available_congestion_control   # reno cubic bbr dgcc
sudo make -C bpf unload
```

Selecting and recording the algorithm. `set_congestion_control()` sets `TCP_CONGESTION` on the data socket before `connect()`. After the transfer, `iperf3_test()` reads it back from the connection and stores it with every sample. All 60 tests used the requested algorithm.

2. Implementation

The HW2 Q3(b) pseudocode:

```
Start up with the minimum window size

ONCE EVERY ROUND TRIP
  1. Measure:
     - base delay    = fastest round trip seen recently
     - current delay = smoothed round trip time
     - extra delay   = how far current delay is above base delay
     - goodput       = how much data got through this round trip

  2. If the connection timed out:
       Shrink window to minimum, go back to start-up mode.

  3. Else if packets were lost:
       If extra delay is large → shrink window to 60% (a real queue built up)
       If extra delay is small → shrink window to 90% (probably random loss)
       Leave start-up mode.

  4. Else if in start-up mode:
       If extra delay is getting noticeable → leave start-up mode
       Otherwise → double the window

  5. Else (normal operation):
       If extra delay is below target:
           Grow the window by a few packets.
           Grow more when far below target, less when close to it.
           If the last increase didn't raise goodput → grow only slightly.
       If extra delay is above target:
           Shrink the window by a percentage.
           Shrink more the further above target it is (never more than half).

  6. Never let the window be smaller than the minimum window size
     or bigger than about twice what the path can hold.
```

2.1 How cwnd is updated from RTT, loss, and throughput

dgcc changes cwnd (measured in segments) once per round trip, called a *round*. The kernel calls `dgcc_cong_control` on every ACK, which records that round's lowest RTT and any losses. When the round ends, it calls `dgcc_round()` to decide the new cwnd.

What dgcc measures each round (L173–L192, L180–L186):

- **Base delay:** the lowest RTT seen recently, i.e. our estimate of the path's delay when no queue has built up. A round with a lower RTT replaces it. If the base delay is more than 10 s old, it is replaced by the current round's lowest RTT, so it can follow a path that changes.
- **Extra delay:** the smoothed RTT (a running average of RTT samples that weights the newest ones most) minus the base delay. This is roughly how long our packets wait in queues.
- **Target:** the extra delay dgcc accepts, equal to the base delay / 8, but at least 2 ms and at most 20 ms. For example, a 110 ms base delay gives a 13.75 ms target.

- **Goodput:** the data acknowledged during the round, divided by the round's length.

How cwnd changes. The rules are checked in this order, and only the first one that applies is used:

1. **Timeout** (`dgcc_set_state`): if the connection times out, cwnd drops to 4 segments and dgcc goes back to start-up. This happens immediately, not at the end of the round.
2. **Loss** (L198–L208): if any packet was lost during the round, cwnd is cut to 60% when the extra delay is above the target (a queue overflowed), or to 90% otherwise (the loss was probably random). Start-up ends.
3. **Start-up** (L209–L214, L264–L267): at the start of a connection, dgcc doubles cwnd every round so it finds the path's capacity quickly. It spreads the doubling over the round: each ACK adds as many segments to cwnd as it acknowledges, growing by a full window by the end. Start-up ends as soon as the extra delay reaches half the target because that indicates a queue is starting to form.
4. **Extra delay below the target** (L215–L221): cwnd grows by 8 segments when there is no extra delay, shrinking linearly to 2 segments as the extra delay approaches the target. **Throughput check** (L218–L219): if the previous round also grew cwnd but goodput did not go up, cwnd grows by only 1 segment, since a larger window is not getting more data through.
5. **Extra delay at or above the target** (L222–L230): the further the extra delay is above the target, the more cwnd is cut. Just above the target the cut is a few percent, at twice the target it is a third, and at three times the target or more it is half, the largest cut allowed. The cut is (extra − target) / (extra + target).

After any decrease, the next round makes no change (see 2.3).

Limits (pseudocode step 6; L232–L240, L275–L279): cwnd never drops below 4 segments. After start-up, it also never goes above about twice what the path can hold, which dgcc estimates as the highest recent goodput × the base delay.

2.2 Initial state and parameters

Initial state (`dgcc_init`): cwnd = 4, start-up mode, base RTT and bandwidth unknown, ssthresh unset.

Parameter (L23–L36)	Value
Minimum window	4 segments
Delay target	base RTT / 8, clamped to [2, 20] ms
Start-up exit	extra delay ≥ target / 2
Loss cut	× 0.6 if extra > target, × 0.9 otherwise
Additive increase	2–8 segments per round, 1 if the previous increase did not raise goodput
Delay-based decrease	(extra − target) / (extra + target), at most 50%
Base RTT window	10 s
Bandwidth window	max round goodput, kept for up to 10 rounds
Upper bound	2 × BDP
Hold after a decrease	1 round

2.3 Changes from the pseudocode

The pseudocode left "large", "noticeable", "a few", "a percentage", and the target unspecified; the parameters above specify what they are. Other changes:

1. Start-up adds to cwnd on every ACK instead of doubling it all at once at the end of a round. It still doubles each round, but avoids sending a big burst.
2. After cwnd is cut, the next round makes no change. The smoothed RTT takes a while to drop after a cut, so without this one congestion event could cause several cuts.
3. If the kernel later decides a lost packet was not really lost, `dgcc_undo_cwnd` puts cwnd back to what it was before the cut.
4. The upper limit (twice what the path can hold) is not used during start-up, because it would stop the doubling too early.
5. dgcc sets the pacing rate itself (L88–L102), since Linux does not do it for algorithms that control cwnd directly.
6. Timeouts are handled as soon as the kernel detects them (`dgcc_set_state`), not at the end of a round.

3. Experiment

- **Algorithms:** dgcc, CUBIC, Reno, BBR.
- **Destinations:** 5 public iperf3 servers drawn at random from `data/listed_iperf3_servers.csv` : 23.106.226.75:5201, 80.67.167.93:9200, 185.93.2.193:5201, 195.162.68.38:5201, 69.48.237.66:30001.
- **Duration and sampling interval:** 10 s per test, sampled every 0.25 s, for all four algorithms.
- **Repetitions:** 3 per algorithm per destination, i.e. **15 tests per algorithm, 60 tests total**.
- **Logged per sample** (`tcp_sample()` , `results/tcp_stats.csv`): `snd_cwnd` (our own congestion window), goodput (Δ `bytes_acked` × 8 / Δt), `srtt` and `min_rtt` , and the loss signal (retransmitted segments per interval).

Test order. `run_destination()` runs all repetitions at one destination before moving to the next. Repetition *r* uses the algorithm list rotated by *r* – *1* (`rep_order()`). Network conditions change over time, so rotating the order keeps any one algorithm from always running at the same point in the sequence. For example, one algorithm may face different conditions because the server is still emptying the queues from the previous algorithm's test, which would result in it receiving an altered state and influence measurements.

Rep	Order
1	dgcc, cubic, reno, bbr
2	cubic, reno, bbr, dgcc
3	reno, bbr, dgcc, cubic

4. Results

4.1 Time series and scatter plots

All plots use the same representative destination, **195.162.68.38:5201**, chosen by `pick_representative()` (median of the per-destination mean goodput). Each figure is one algorithm with one line per repetition, and every axis has the same range in all four algorithms' figures (`analyze()`).

TCP stats over time (195.162.68.38:5201)

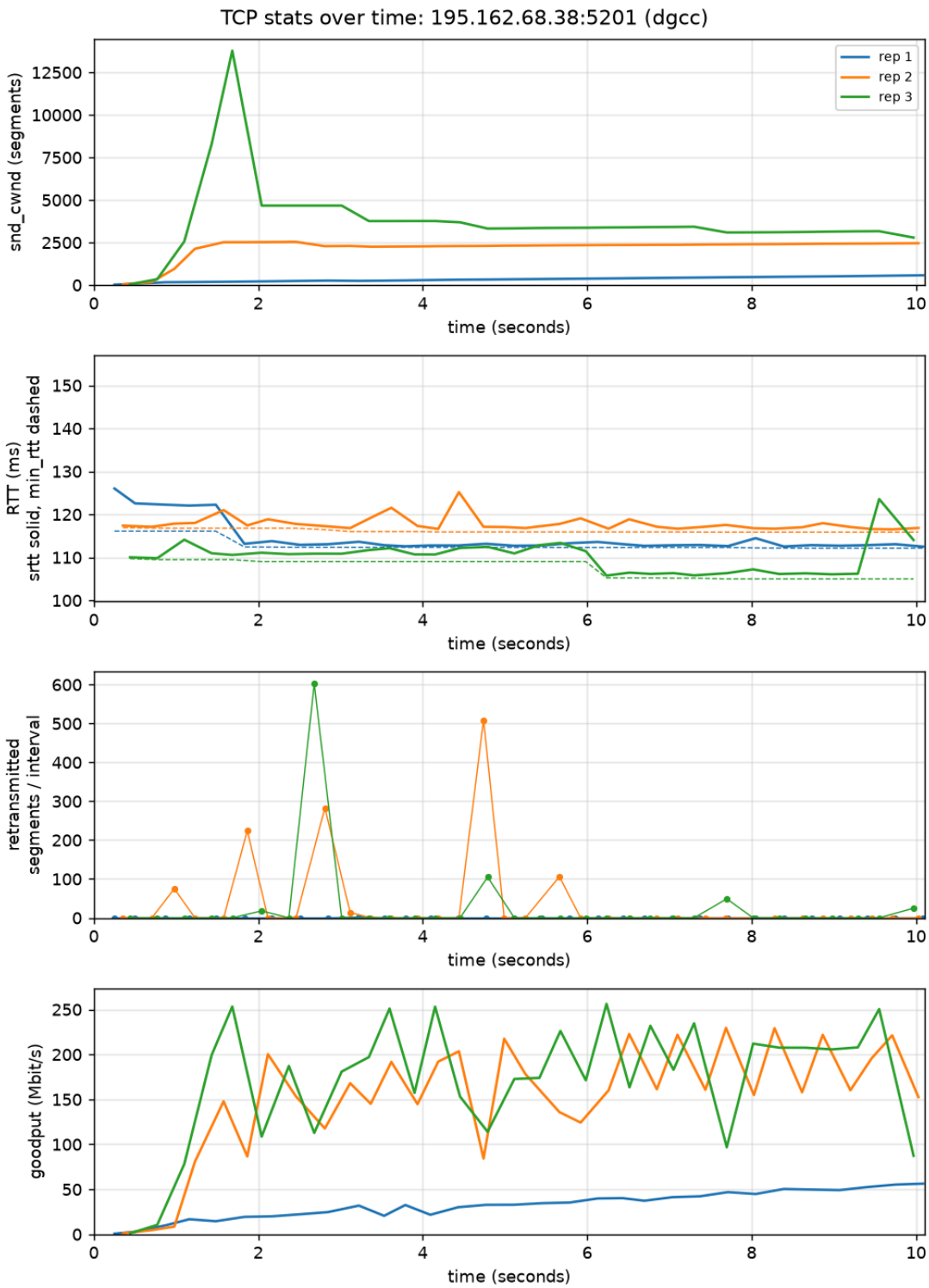


Figure 1a: dgcc. Generated by `plot_tcp_stats()`.

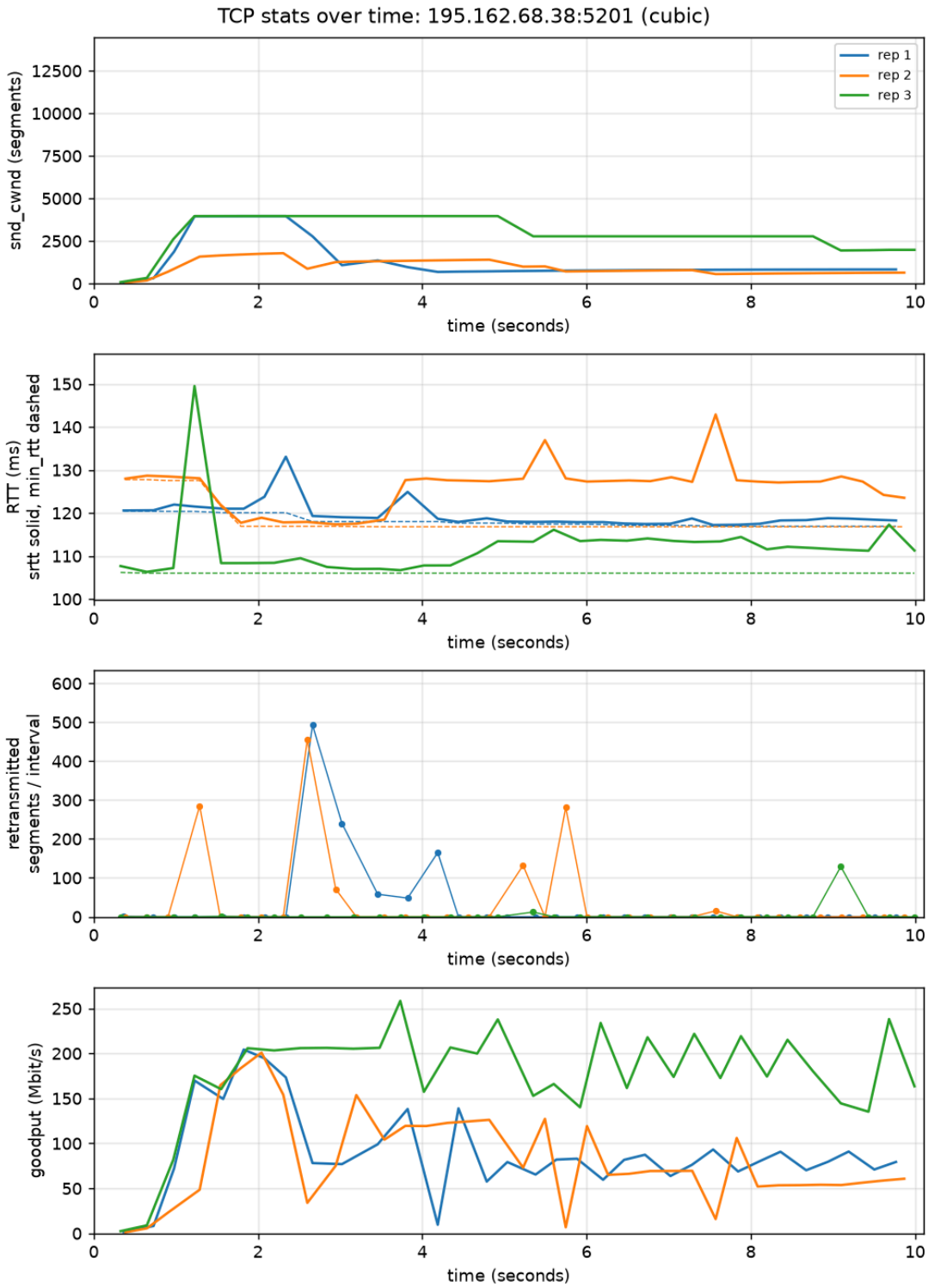


Figure 1b: CUBIC. Generated by `plot_tcp_stats()`.

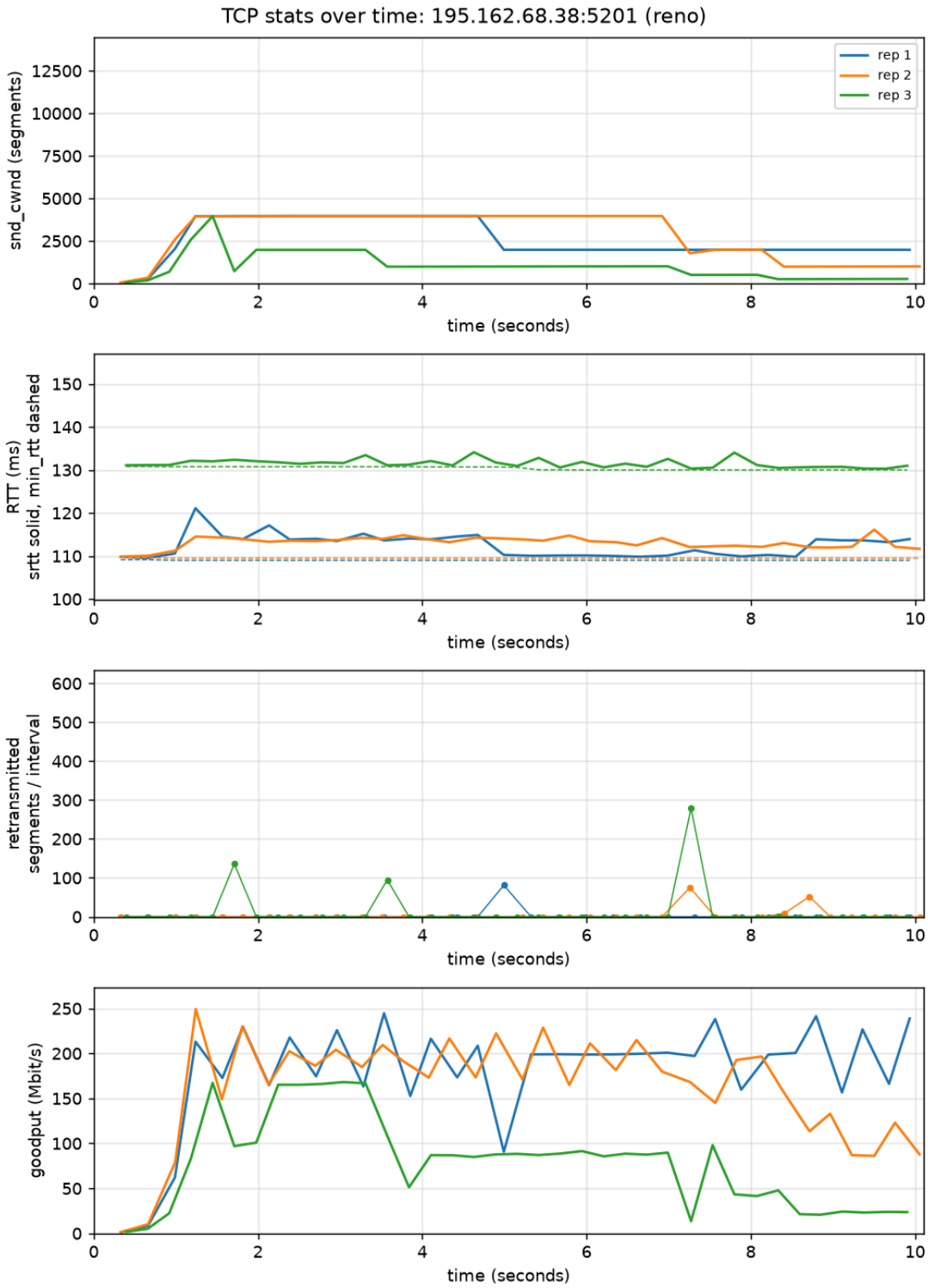


Figure 1c: Reno. Generated by `plot_tcp_stats()`.

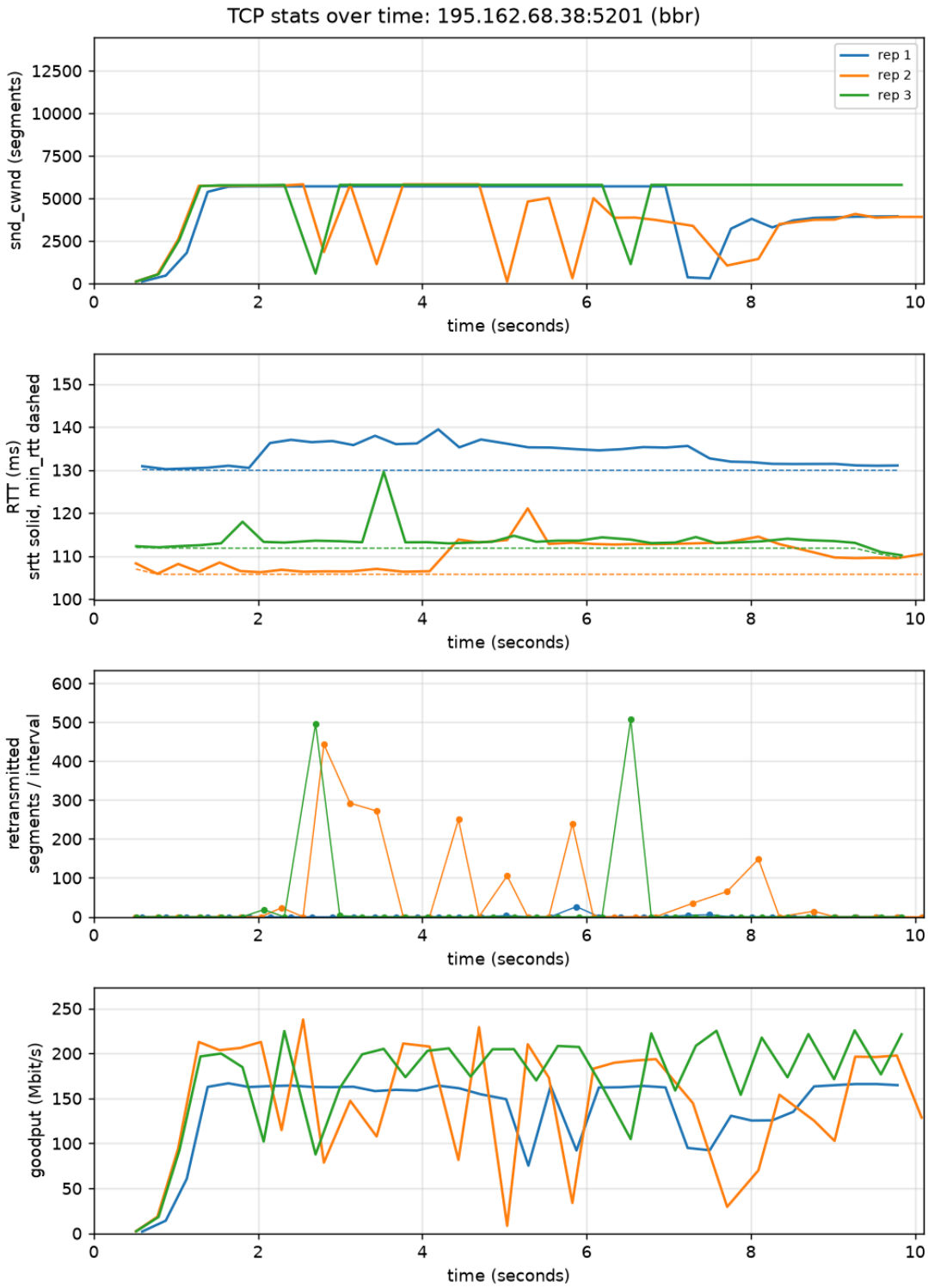


Figure 1d: BBR. Generated by `plot_tcp_stats()`.

TCP stats vs goodput (195.162.68.38:5201)

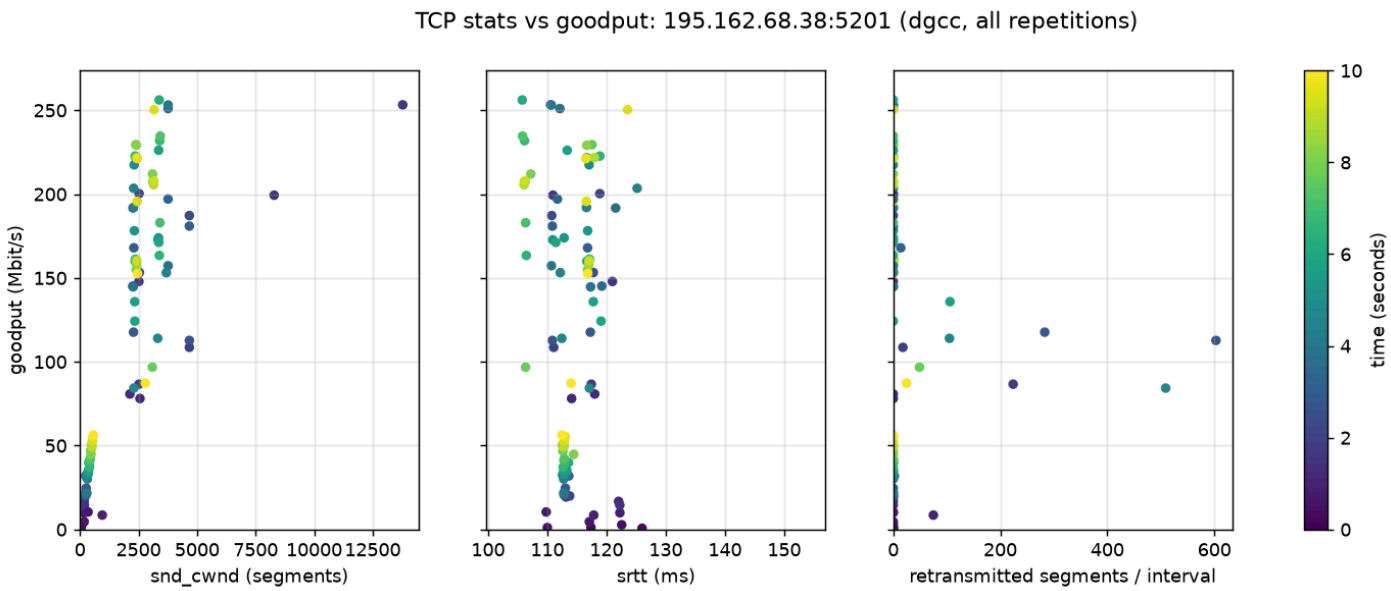


Figure 2a: dgcc. Generated by `plot_tcp_stats()`.

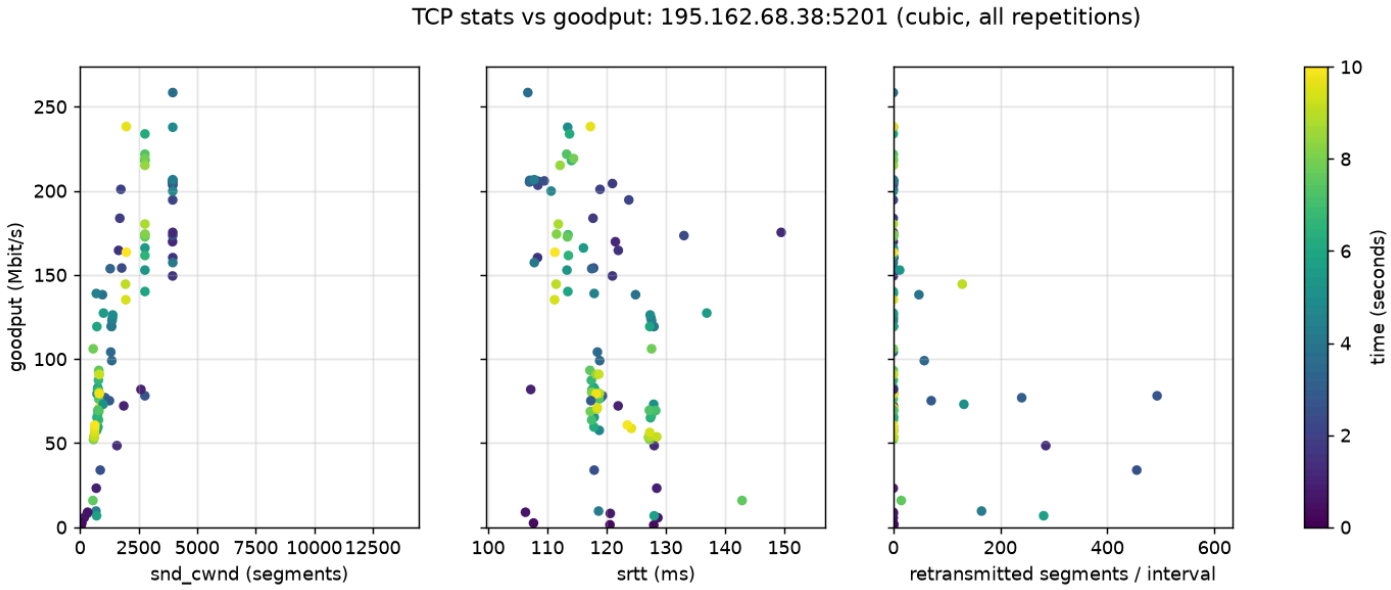


Figure 2b: CUBIC. Generated by `plot_tcp_stats()`.

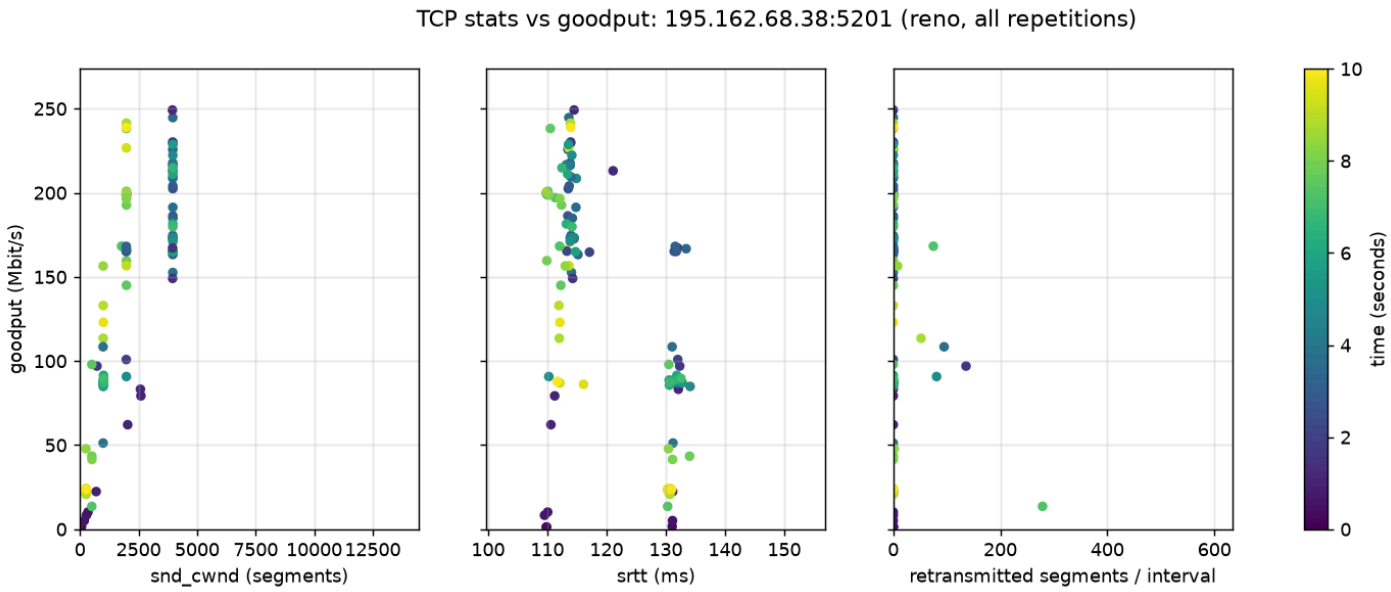


Figure 2c: Reno. Generated by `plot_tcp_stats()`.

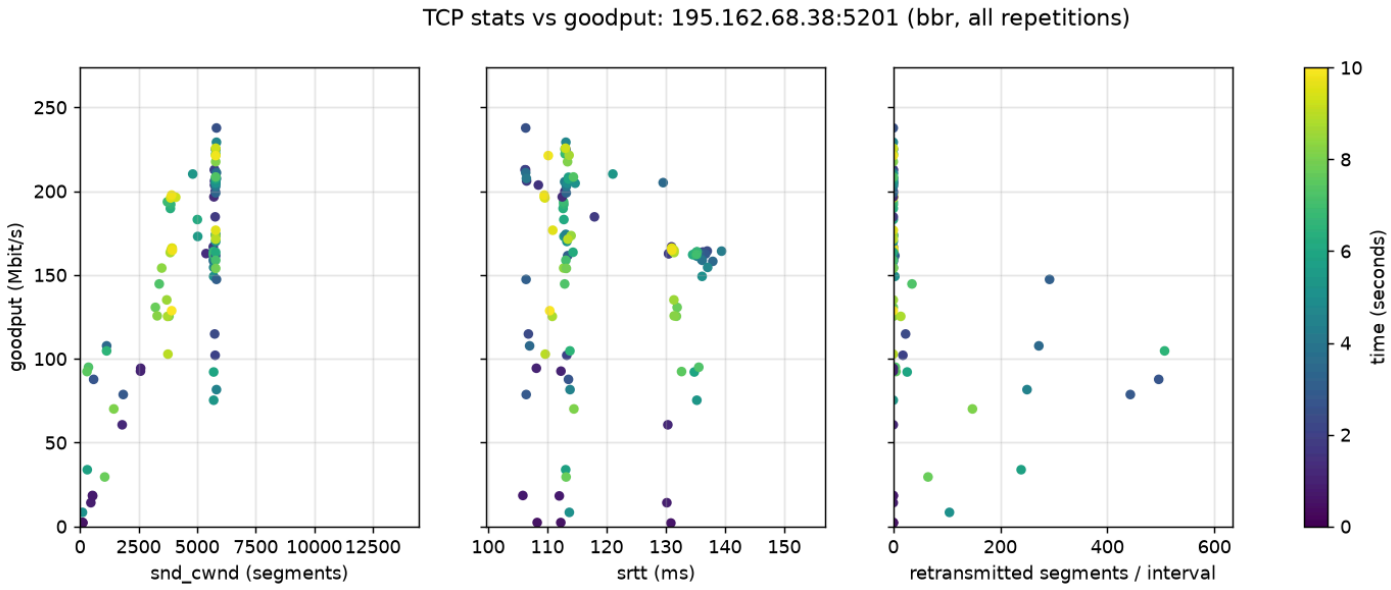


Figure 2d: BBR. Generated by `plot_tcp_stats()`.

4.2 Aggregate statistics

Over all samples from 5 destinations × 3 repetitions (`results/aggregate.csv` , computed by `summarize()`). Queueing delay is srtt − min_rtt for each sample.

	dgcc	CUBIC	Reno	BBR
goodput mean (Mbit/s)	129.4	141.0	117.1	151.4
goodput median (Mbit/s)	153.2	155.8	119.8	158.9
goodput p95 (Mbit/s)	237.5	228.4	225.8	237.0
srtt mean (ms)	119.2	120.3	118.9	124.1
srtt median (ms)	113.9	118.7	114.2	113.3
srtt p95 (ms)	144.9	143.3	144.2	149.6
queueing delay median (ms)	1.7	1.6	1.5	1.0
queueing delay mean (ms)	3.0	4.0	2.7	7.2
queueing delay p95 (ms)	10.1	10.5	4.9	8.0
retransmitted segments per test (mean)	379	569	511	711

4.3 Discussion

Metric	Meaning	Expected effect on goodput in congestion avoidance
snd_cwnd	Max number of unacknowledged segments the sender can have in flight	Goodput ≈ cwnd × MSS / RTT until cwnd reaches the BDP. Past that, extra cwnd just fills the bottleneck queue.
srtt	Smoothed RTT estimate	For a fixed cwnd, higher RTT means lower goodput. srtt rising above min_rtt means packets are queueing.
Retransmitted segments	Our loss proxy	Each loss event makes our algorithm dynamically cut the cnwd based on the amount of extra delay.

On every path, packets were often lost (about 500 retransmitted segments per test) but rarely delayed by queueing, so throughput depended mostly on how each algorithm adjusted cwnd based on loss. BBR does not slow down on loss (keeps the sending rate the same), so it had the highest throughput. Reno halves cwnd on each loss and regains only one segment per round trip, so it never recovered within the 10 s tests and had the lowest. With longer tests, Reno would likely have had time to rebuild cwnd and come closer to the others. However, Reno did have the second-fewest retransmitted segments, showing that adjusting cwnd in a controlled and slow manner does have some positive tradeoffs.

CUBIC shrinks cwnd less and regrows faster, so it fell in between. dgcc matched or beat CUBIC in 11 of 15 tests thanks to its smaller cut on random loss, but in the other 4 its start-up ended early and it grew too slowly to reach full speed, which lowered its average. RTT was similar for all four algorithms because most of it is the travel time to the server (about 110 ms), which no algorithm can change. An algorithm only adds delay by

building a queue, and every algorithm's queueing delay stayed small (1–2 ms median). BBR's slightly higher average comes mostly from one test where its RTT spiked above 1 s before the connection timed out.

It's important to note that this could change if queueing delay became larger, because the algorithms have different ways of adjusting cwnd. For example, our algorithm adjusts cwnd based on how much queueing delay there is (larger queueing delay results in a larger decrease of cwnd).

AI use

Claude Code (Claude Opus) was used to:

- write the eBPF implementation from our pseudocode;
- extend the HW2 client for four algorithms, repetitions, and aggregate statistics;
- run the experiments;
- draft this report.

We reviewed the code and checked the analysis against the raw data.